Atmosphere AI Project Proposal

Air quality analysis and hourly fine particulate matter forecasting

26 October 2026 Version 1.1 English edition

Executive summary

Atmosphere AI is a proposed web platform that brings together air quality data, historical trends and classical machine learning to forecast hourly PM2.5 concentrations for the next 72 hours. It serves people exploring conditions in a particular city and gives students and researchers access to the underlying data, evaluation methods and model performance.

This proposal builds on an implemented prototype connecting a user interface, backend service, database and reproducible training pipeline. It defines the product scope, initial feasibility evidence and a proposed plan to improve evaluation, coverage and operation. Reported results come from the documented experiment of 15 October 2026; they are not a fresh measurement of system performance on the proposal date.

Problem statement

Air quality information is often presented as isolated readings or spread across sources and pages. Users struggle to connect a concentration with its timestamp and recent trend, or understand whether pollution may rise over the next three days. Researchers also need traceable data and an inspectable comparison between predictions and baselines. The specific problem is the lack of context, traceability and evaluable forecasting within one experience.

Why it matters

Combining readings, provenance, history and forecasts can help people understand changes and make more informed plans for outdoor activity. Students and researchers can study environmental data and evaluate forecasting models through a reproducible workflow. These are intended benefits to validate with users, not demonstrated reductions in pollution exposure. The platform does not provide official health alerts or medical advice.

Objectives

- Support city search and display available pollutant data with timestamps, provenance and explicit missing-data states.
- Generate 72 hourly PM2.5 predictions using a model trained within the project, preserving model versions and their results.
- Compare classical machine learning algorithms against simple baselines using chronological splits that prevent target overlap.
- Provide maps, charts and city comparisons alongside tools for inspecting data quality and model performance.
- Establish a reproducible foundation for further development, clearly distinguishing implemented functions from proposed improvements.

Target users and team roles

Target users

User group	Need	Platform response
Residents and commuters	Understand changing air conditions and plan outdoor activities	City search, timestamped readings, history and forecasts for supported locations
Students and researchers	Inspect data, compare models and understand evaluation limits	Documented sources, coverage, missingness, metrics and training settings
Environmental analysts and educators	Compare cities and explain trends with visible data-quality context	Explorable charts and comparisons across two to five cities

Team roles and responsibilities

The following is a proposed allocation of responsibilities. One person may cover multiple roles. Assignments will be made when the phase begins; this proposal assumes neither named members nor a fixed team size.

Role	Main responsibility	Owned deliverable
Project coordinator	Set priorities and scope, track milestones and coordinate handover	Work plan, acceptance checklist and progress report
Data engineer	Collect data, validate timestamps, missingness and provenance, and preserve snapshots	Documented dataset snapshot and quality report
ML engineer	Prepare features, train models and baselines, evaluate and calibrate	Versioned model and reproducible evaluation report
Frontend developer	Implement city journeys, charts, comparisons, forecasts and mobile layouts	Connected screens with clear loading and error states
Backend and operations developer	Own APIs, persistence, scheduling, deployment and backups	Reliable service and operating and recovery plan
QA and documentation owner	Test complete flows and review claims, documentation and usability	Acceptance evidence and user documentation

Coordination and handoffs

The data engineer supplies a fixed snapshot to the ML engineer, who supplies the model and input/output contract to the backend developer. The frontend developer connects this data to the user journey. QA checks the complete flow before the coordinator accepts the deliverable. Acceptance review includes gathering primary-user feedback on city selection, timestamps and the distinction between history and forecasts.

Solution scope and technical architecture

User experience

Users select a city through search or the map, then review the latest available readings and historical trends. The forecast screen distinguishes past data from predictions and lets users inspect individual hours. Users can compare two to five cities or examine dataset coverage, training results and system status.

Current scope and proposed development

Area	Implemented prototype	Proposed development
City exploration	Worldwide search, maps, readings and history where source data exists	Broader coverage and source-quality monitoring
Forecasting	72-hour PM2.5 forecasts for seven training cities	Extend coverage after independent evaluation
Analysis	City comparisons, temporal patterns and dataset inspection	Expand historical weather-data exploration
Model evaluation	Measured metrics and global feature importance	Calibrated prediction intervals and prospective operational evaluation
Operation	Database, scheduling and Docker configuration	Public hosting, monitoring and backups

Technical architecture

The frontend uses Next.js, React and TypeScript, with MapLibre for maps and ECharts for charts. It communicates with FastAPI, which validates requests and coordinates access to upstream sources and PostgreSQL. SQLAlchemy and Alembic manage persistence and schema migrations.

A separate pipeline collects data, prepares features and compares Ridge Regression, Random Forest, LightGBM and XGBoost. It saves the model and metadata as an immutable version, while the database records the version used for each forecast run. Training is an operator task separate from user requests. A scheduler refreshes configured cities.

Outside the current scope

The prototype does not forecast PM10 or calculate an official AQI, and forecast coverage does not extend to every searchable city. It does not infer a continuous worldwide pollution heatmap from sparse points or present reliable prediction intervals before calibration. Weather data provides user context but is not an input to the current forecasting model.

Data methodology and feasibility results

Data provenance and sample

Air quality inputs are Copernicus CAMS estimates served through Open-Meteo. They are environmental model outputs, not direct ground-station measurements. The documented experiment contains 70,032 hourly air quality records and the same number of weather records across Cairo, Delhi, London, New York, São Paulo, Sydney, Tokyo and Nairobi. Coverage runs from 16 October 2025 through part of 15 October 2026.

Training and evaluation

Inputs include lagged pollution readings, rolling averages and trends, time, coordinates and the requested horizon. The model does not use future realized weather. Inference requires 73 consecutive nonmissing PM2.5 hours and rejects requests with insufficient data or unsupported locations.

Training uses one pooled direct model for multiple horizons, fitting ten selected horizons to limit cost. Validation, testing and inference cover every horizon from 1 to 72 hours. Known cities are split chronologically into training, validation and test periods, with a full 72-hour purge at the boundaries. Nairobi is excluded from training and model selection to evaluate an unseen city.

Documented initial results

Measure	Result	Interpretation
Selected model	LightGBM	Lowest validation MAE among trained estimators
Test MAE	8.5785 μg/m³	Later period in known cities
Test RMSE	14.3597 μg/m³	Greater sensitivity to large errors
Persistence MAE	10.2690 μg/m³	Last-reading baseline on the same test set
Nairobi MAE	4.0520 μg/m³	City excluded from fitting and selection

Interpretation and limitations

LightGBM reduced test MAE by approximately 16.46% relative to persistence. Selection used validation results, so XGBoost did not replace it despite a slightly lower test error. These metrics are not accuracy percentages or guarantees for individual cities.

Evaluation uses retrospectively retrieved archives without verified historical publication times or revisions. It therefore does not establish performance as if forecasts had been issued at the historical time. Different pollution distributions also mean Nairobi's lower raw error alone is not evidence of worldwide accuracy.

Development plan resources and risks

Proposed implementation plan

This proposed four-week plan extends the existing prototype. Durations are estimates starting after scope approval and resource availability, not a delivery commitment. Extended operational evaluation or integration with new monitoring stations may take longer.

Period	Work	Owner	Phase output
Week 1	Freeze data; check quality and reproducibility	Data engineer and QA	Dataset snapshot and quality report
Week 2	Evaluate cities, horizons and concentration bands	ML engineer	Evidence-based coverage decision
Week 3	Trial interval calibration and weather-data exploration	ML engineer and frontend developer	Coverage report and UI improvements
Week 4	Pilot deployment, monitoring and backups	Backend/operations developer and QA	Pilot environment and acceptance report

Resources and cost

Delivery requires frontend, API, data management and machine learning skills; one person may cover several roles. Resources include a CPU-based training computer, storage for data and model versions, and PostgreSQL, Python and Node.js runtimes or Docker. The current approach does not depend on GPU-intensive deep learning.

Budgeting depends on hosting, storage, refresh frequency and appropriate access to data providers. This proposal contains no price quotation or confirmed monthly cost. Provider terms and licensing should be checked before commercial operation or wider deployment.

Risks and proposed responses

Risk	Proposed response
Unavailable or stale source data	Bounded retries, visible timestamps and rejection of invalid forecast requests
Poor transfer to a new city	Independent evaluation and explicit coverage limits before enabling forecasts
Distribution shift or rising error	Retain results, monitor errors and assess the need for retraining
Unreliable prediction intervals	Separate calibration from testing and measure coverage before display
Multiple service instances	Coordinate scheduler ownership, shared rate limits and consistent backups

Deliverables acceptance criteria and references

Target deliverables

- A web application connecting city exploration, pollutant history, city comparisons and forecasting.
- An API and database preserving readings, ingestion jobs, model versions and forecast runs.
- A reproducible collection, training and evaluation pipeline with documented data, metrics and experiment settings.
- An evaluation report by city and horizon, including coverage limits, uncertainty and the model-selection rationale.
- Startup and deployment configuration, user and maintenance documentation, the overview presentation and proposal files.

Acceptance criteria

- A successful forecast request returns 72 consecutive, timestamped, finite and nonnegative predictions with model version and source-data time.
- The application explains missing data and unsupported cities, without substituting fabricated environmental readings.
- Evaluation compares the model with two baselines on identical periods, uses validation alone for selection and purges overlapping targets.
- Results are traceable to the data snapshot, training configuration and model version; results that do not beat baselines are reported openly.
- Search, history, forecasting and comparison flows work on desktop and mobile, including error handling and retry states.
- Future coverage or prediction-interval additions require independent evaluation evidence; success is not assumed in advance.

Existing implementation evidence

Implementation records dated 15 October 2026 report 25 passing backend and ML tests, 14 frontend tests and seven browser journeys against both native and Docker deployments. These are historical prototype results, not claims that tests were rerun for this proposal. The source code is public on GitHub; public application hosting remains a separate task.

Project references

1. Public repository
2. Original build requirements
3. Training and evaluation methodology
4. Saved experiment results
5. Implementation status and remaining work
6. Data sources and attribution